

Operating Systems Project — Performance Analysis Report

Generated: 2026-07-14 02:49:49

1. System Configuration

Parameter	Value
OS	Windows 10
Python	3.14.0
CPU cores (logical)	8
Worker processes used	8
Images processed	45
Batches discovered	batch_A, batch_B, batch_C
Pipeline	load -> grayscale -> blur (r=1) -> edge detection on blurred -> save

2. Design Justifications

Why multiprocessing instead of threading?

Image filtering in Pillow is CPU-bound Python work. The Global Interpreter Lock (GIL) prevents multiple threads from executing Python bytecode in parallel, so threads would not speed up the blur and edge-detection steps. Separate processes each have their own interpreter and can run on different CPU cores at the same time.

Why this worker count?

The default configuration uses 8 worker processes, matching the 8 logical CPU core(s) reported by the system. This aims to keep each core busy without spawning so many processes that scheduling and IPC overhead dominate. The `--workers` flag allows tuning when tasks are heavier or I/O-bound.

Why Queue, Lock, and Semaphore?

Queue — safely distributes (input, output) task pairs from the main process to workers and collects result dictionaries without manual shared-memory management. Sentinel values (`None`) signal workers to exit cleanly.

Lock — serializes console logging so lines from parallel workers do not interleave and become unreadable.

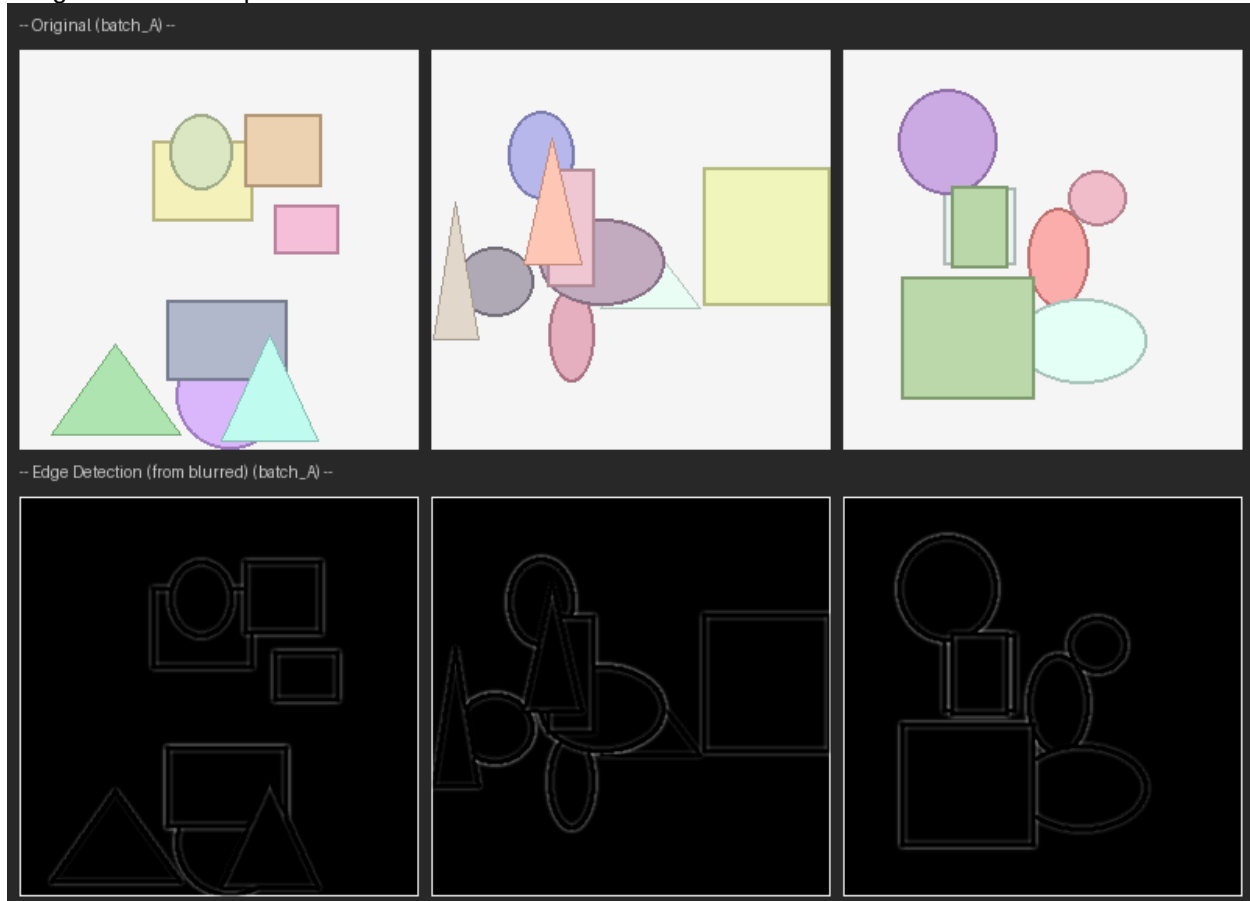
Semaphore(2) — limits how many workers perform disk reads/writes at once, reducing I/O contention when many processes share one disk.

3. Concurrency Primitives

Primitive	Role
Process	Workers run the CPU-bound filter pipeline in parallel
Queue	Task distribution and result collection between processes
Lock	Protects shared console output

4. Visual Comparison (Original vs Edge-Detected)

Sample images from the first discovered batch showing the original input alongside the processed edge-detection output.



5. Benchmark Results

Mode	Workers	Time (s)	Speedup	Efficiency (%)
sequential	1	0.2872	1.0	100.0
parallel	2	0.5406	0.531	26.6
parallel	4	0.6363	0.451	11.3
parallel	8	0.9045	0.318	4.0

Sequential baseline: 0.287 s for 45 images (6.4 ms/image).

6. Performance Analysis

Parallel runs were slower than sequential on this machine (best: 2 workers at 0.531x of baseline). Process spawn and Queue IPC overhead can exceed compute savings for small images, especially on Windows where the default start method is spawn.

7. Output Files

Processed edge images: output/<batch>/

Per-image statistics: output/image_stats.csv

Analysis summary: reports/analysis_summary.txt

Raw benchmark JSON: output/benchmark_results.json